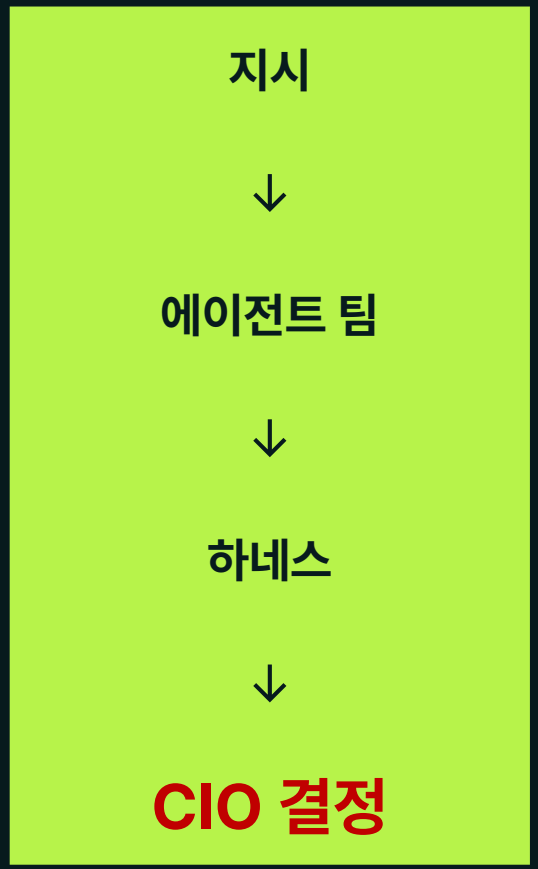
기말 프로젝트를 위한 AI 기초

Claude Code · 멀티에이전트 · 하네스를 한 번에 정리합니다



XS 팀 프로젝트 · 가이드 겸 치트시트

필수 지식은 네 묶음이면 충분합니다

01	역할 나누기	**누가 무엇을 하는가** 사람 · Claude Code · 에이전트 · 용어 다섯 개
02	Claude Code 다루기	**폴더를 열고 말로 시킨다** 작업 고리 · 폴더 지도 · 지시의 네 칸
03	멀티에이전트	**에이전트 13개가 한 분기를 돈다** 에이전트 파일 · 분기 파이프라인 · 경쟁과 투표
04	하네스와 감독	**계산은 코드, 결정은 사람** 역할 분담 · 킬스위치 · LLM 약점 · 치트시트

코딩 실력보다 '무엇을 시키고 어떻게 확인하는가'가 중요합니다

여러분은 코드를 쓰지 않고 지시하고 확인합니다

사람 (팀원 · CIO)

지시 · 확인 · 승인

- 만들 것을 말한다
- [확인]으로 점검한다
- CIO로 결정한다

판단의 주인



Claude Code

작성 · 실행 · 수정

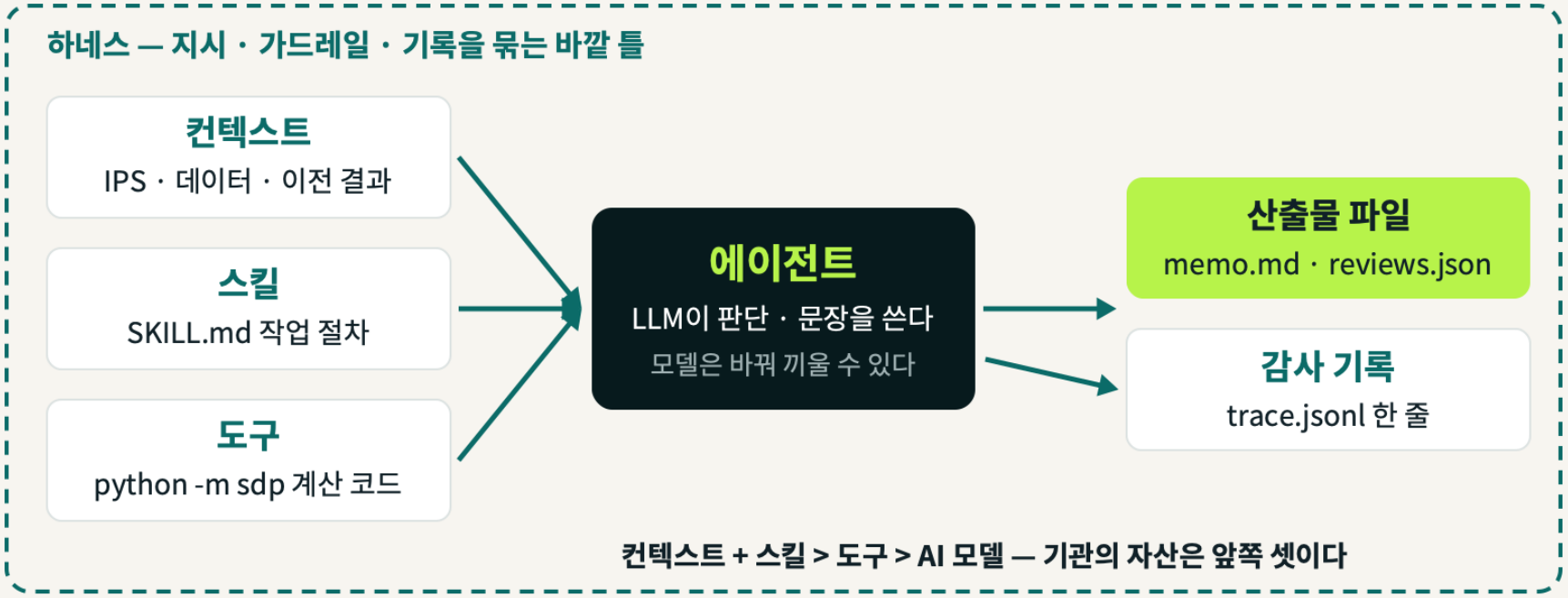
- 코드를 쓴다
- 파일을 고친다
- 점검을 돌린다

손이 되는 에이전트

숫자가 맞는지는 반드시 사람이 확인합니다 — M8 검산 예제가 그 연습입니다

에이전트는 컨텍스트·스킬·도구를 받아 판단하는 AI 작업자입니다

하네스가 바깥에서 지시 · 가드레일 · 기록을 묶는다



모델은 바꿔 끼울 수 있지만 IPS · 절차서 · 실행 기록은 기관 고유의 자산입니다

다섯 개의 용어만 알면 과제문이 읽힙니다

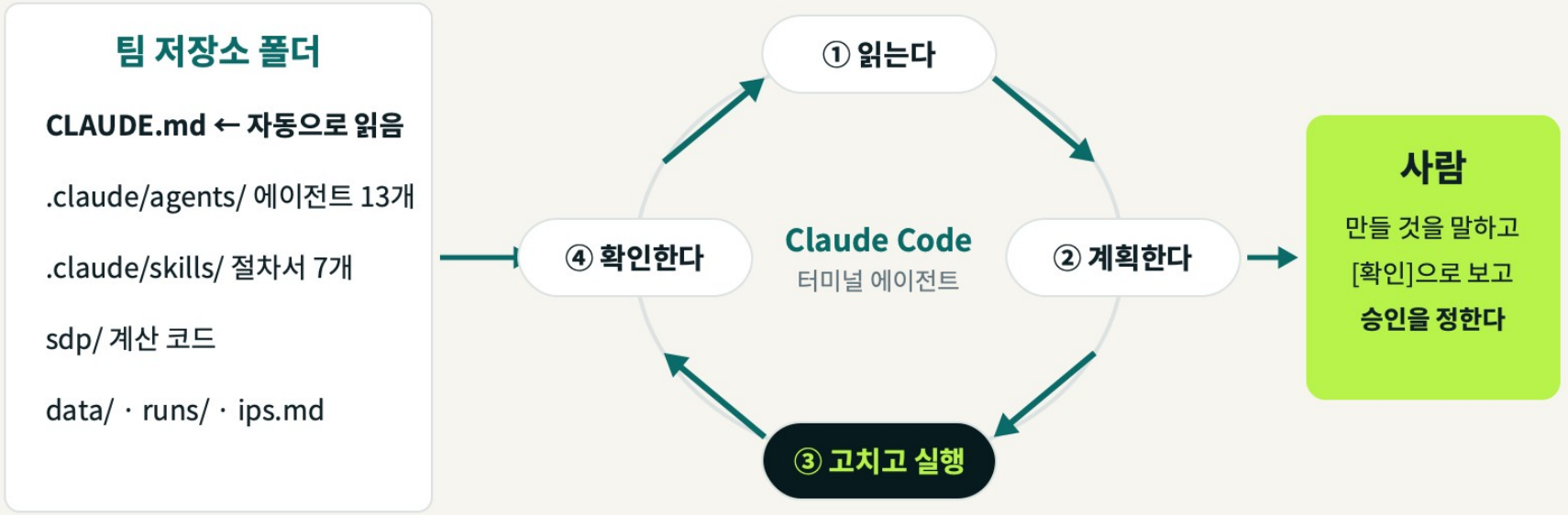
Ang · Azimbayev · Kim(2026)의 용어를 그대로 쓴다

용어	뜻	예시 MVP에서
컨텍스트	에이전트가 참고하는 정보 전체	ips.md · data/ · 직전 분기 결과
스킬	업무 순서를 적은 절차서 한 장	.claude/skills/*/SKILL.md 7개
도구	에이전트가 부르는 계산 코드	python -m sdp ...
하네스	지시 · 가드레일 · 기록을 묶는 틀	자동 기각 · 보호 파일 · trace
오케스트레이터	에이전트를 순서대로 부르는 진행자	quarterly-loop 스킬 · pipeline.py

노력의 대부분은 모델이 아니라 연결과 평가, 곧 하네스에 들어갑니다

Claude Code는 폴더를 읽고 계획하고 실행한 뒤 확인합니다

팀 저장소 폴더를 Claude Code로 열면 CLAUDE.md를 먼저 읽는다



터미널 명령은 치지 않습니다 — 폴더를 열고 순서대로 말로 시킵니다

폴더의 다섯 자리를 알면 정확히 시킬 수 있습니다

예시 MVP(3_Example_MVP/)와 팀 저장소의 구조는 같다

자리	무엇이 있나	누가 고치나
CLAUDE.md	Claude Code가 지킬 하네스 규칙	팀 (처음에 한 번)
.claude/agents/	에이전트 정의 파일 13개 + 접수	팀 · Claude Code
.claude/skills/	분기 루프 · 투표 등 절차서 7개	CIO 승인 후에만
sdp/	계산 코드 · 오케스트레이터 · 점검	Claude Code
ips.md · config/	정책과 파라미터 (보호 파일)	CIO 승인 후에만

보호 파일을 에이전트가 고치게 두면 감독 점수를 잃습니다

좋은 지시는 어디·입력·출력·확인의 네 칸으로 씁니다

과제 가이드 §10의 시작 키트 P1~P9가 모두 이 형식이다

칸	무엇을 적나	예 — 시작 키트 P1
어디	작업할 폴더	예시 MVP zip을 푼 팀 저장소
입력	시킬 일 한두 문장	채점 콘솔을 열고 T1~T8을 돌려줘
출력	생길 파일 · 화면	채점 콘솔, reports/selfcheck.md
확인	성공이면 보이는 것	8/8 통과, 채택 경로 연율 7.90%

확인 칸이 없으면 일이 끝났는지 알 수 없습니다.

모든 지시에 '무엇이 보이면 성공인가'를 함께 적습니다

에이전트 하나는 마크다운 파일 하나로 정의됩니다

.claude/agents/ 안의 파일을 Claude Code가 읽고 그 역할을 맡는다

.claude/agents/pc-erc.md

```
---
name: pc-erc
description: 위험기여 균등 안
---
# 하는 일
위험기여 균등 배분안을 낸다
# 일하는 방식
최적화는 sdp/agents_pc.py가 한다
# 산출물
proposals/pc-erc.json
# 하지 않는 일
숫자를 직접 계산하지 않는다
```

- → 이름 — 부를 때 쓰는 이름
- → 설명 — 언제 이 에이전트를 쓸지
- → 역할 한 줄
- → 계산은 코드에 맡긴다
- → 결과 파일의 자리
- → **금지선 — 가장 중요한 칸**

새 에이전트는 기존 파일을 복사해 칸만 바꾸라고 Claude Code에게 시킵니다

분기마다 에이전트 13개가 정해진 순서로 일합니다

오케스트레이터(quarterly-loop 스킬)가 차례로 부르고 결과를 모은다



배분안 6개가 경쟁하고 서로 두 건씩 심사한 뒤 투표합니다 — 혼자 정하면 자기 안을 편애합니다

계산은 코드, 판단은 에이전트, 결정은 사람이 맡습니다

채점자는 코드 계산을 다시 하고, 에이전트 문장은 녹화본으로 재생한다

무엇	누가	채점자는
기대수익 · 공분산 · 최적화 · 위험 지표	코드	다시 계산 — 소수 6자리 일치
투표 점수 집계	코드	다시 계산
매크로 해석 · 리서치 메모	에이전트	녹화본 재생 — 양식과 근거
검토 의견 · IC 보고서	에이전트	녹화본 재생 — 양식과 근거
CIO 결정 · 변경 승인	사람	csv 입력으로 재생

숫자 계산을 LLM에게 맡기면 계산 재현 점검(T6)에서 떨어집니다

하네스의 네 규칙과 킬스위치가 에이전트를 감독합니다

에이전트는 규칙을 직접 고칠 수 없고 제안서만 낸다

숫자는 코드로만
에이전트는 계산하지 않는다

보호 파일
ips.md · config · skills

시점 잠금
as_of 이후 자료 금지

CIO는 사람
IC 보고서에서 멈춘다

규칙을 바꾸고 싶을 때 — 킬스위치

에이전트
수정을 제안

→

proposals/
제안서로만 남김

→

CIO 승인
approvals.csv

→

테스트 통과
T1~T8

→

반영
CHANGELOG.md

모든 호출은 runs/{as_of}/trace.jsonl 에 한 줄
누가 · 어떤 입력으로 · 어떤 스킬로 · 무엇을 냈고 · 어떤 제약이 걸렸나

meta-reviewer에게 '지시문을 고쳐줘'라고 해도 파일이 바뀌지 않아야 합니다

LLM의 약점 다섯 가지를 하네스가 하나씩 막습니다

약점을 알면 어떤 감독 장치가 왜 필요한지 설명할 수 있다

약점	증상	이 프로젝트의 대응
환각	없는 수치 · 출처를 지어낸다	숫자는 코드 · 근거 파일 기록
비결정성	같은 지시에 다른 문장	녹화본 재생 · 3회 재실행
미래 정보 누설	as_of 이후 자료를 쓴다	시점 잠금 · 날짜 검사
자기 수정	IPS · 지시문을 바꾼다	보호 파일 · 킬스위치
과신 · 동조	권고를 그대로 받아들인다	적대적 분산자 · CIO 사유 기록

불리한 결과를 숨기면 감점입니다 — 감독 장치가 놓친 것도 보고합니다

막히면 이 다섯 문장부터 Claude Code에게 말합니다

과제 가이드 §10 시작 키트의 핵심만 줄였다

하고 싶은 일	Claude Code에게 이렇게
출발점 확인	**채점 콘솔을 열고 공개 점검 T1~T8을 돌려줘**
판단을 LLM으로	**research-agent 대역을 에이전트로 바꾸되 숫자는 코드가 만들게 해줘**
감사 기록	**모든 에이전트 호출을 runs/{as_of}/trace.jsonl에 기록해줘**
분기 루프	**2022-09부터 분기마다 돌리고 IC 보고서가 나오면 멈춰줘**
CIO 화면	**이번 분기에 내가 판단해야 할 것만 한 화면으로 정리해줘**

결과가 이상하면 '왜 이렇게 됐는지 근거 파일과 함께 설명해줘'라고 묻습니다

계산은 코드

판단은 에이전트

결정은 사람

모르는 것은 Claude Code에게 묻고,
맞는지는 [확인]으로 본다

자세히: XS_Team_Project_Guide.pdf §1 기본 지식 · §10 시작 키트 · 예시 MVP의 CLAUDE.md